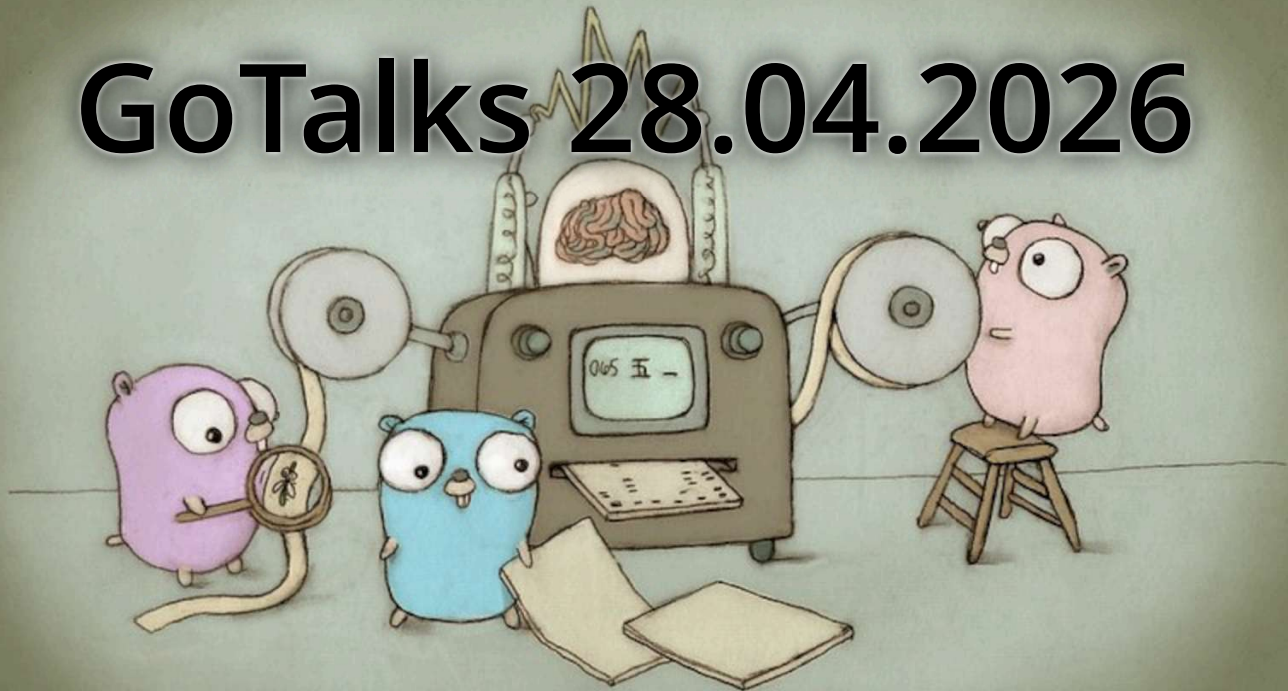


GoTalks 28.04.2026



How to reach us



meetup.com/Golang-ZG



[@golangzg](https://www.youtube.com/@golangzg)



github.com/golang-zg/meetups



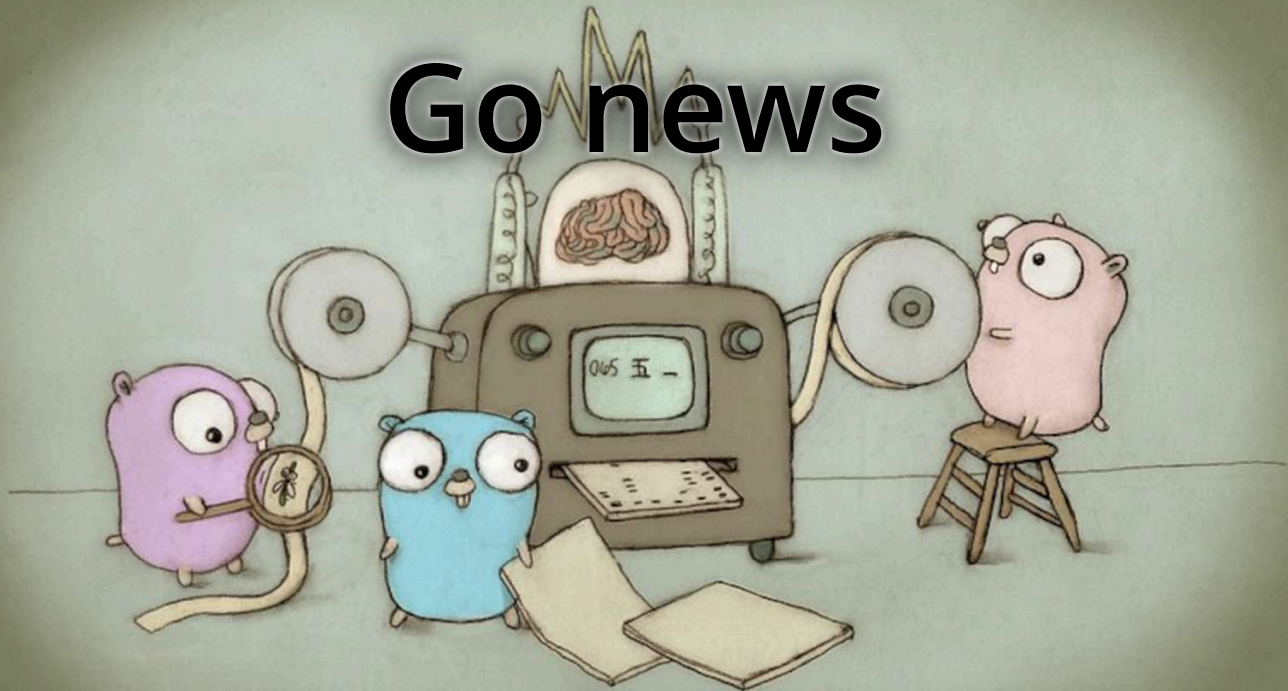
[@golangzg.bsky.social](https://bsky.app/@golangzg.bsky.social)



invite.slack.golangbridge.org



Go news



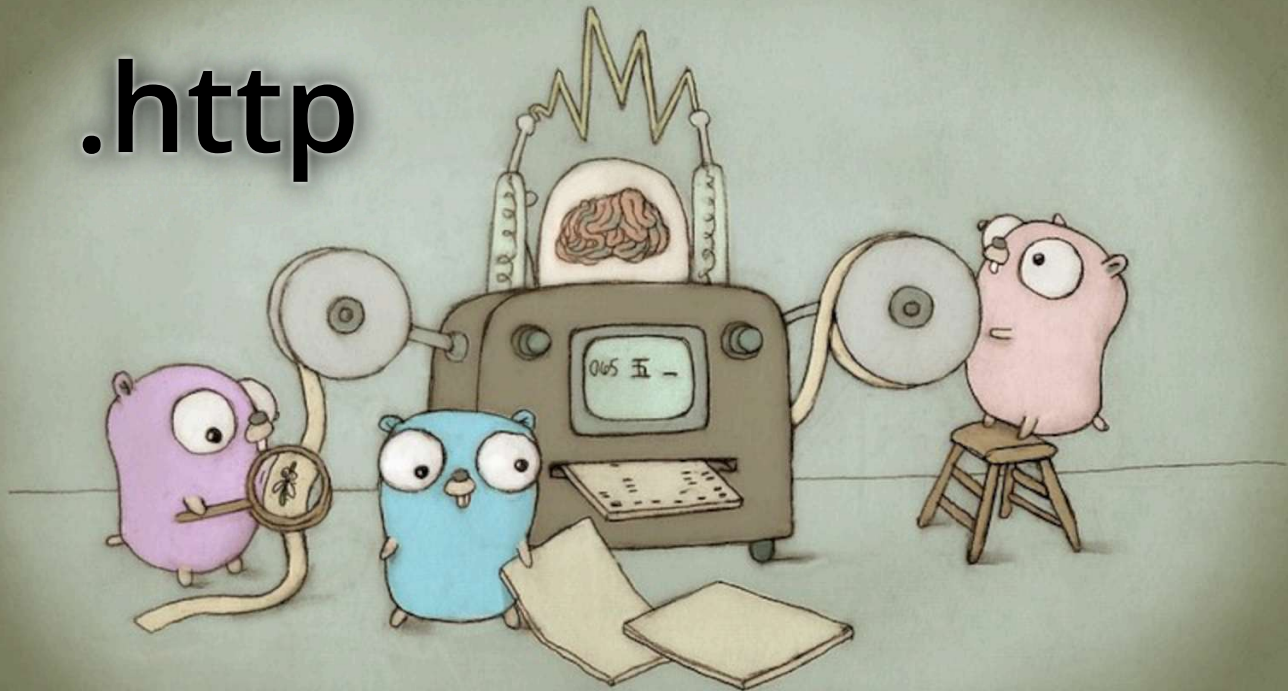
Accepted

- spec: generic methods for Go go.dev/issue/77273
 - allows defining methods on generic types, and using type parameters in method signatures
- spec: direct reference to embedded fields in struct literals go.dev/issue/9859
 - allows struct literals to directly initialize fields of embedded structs without needing to specify the embedded struct's name
- runtime/pprof, runtime: new goroutine leak profile go.dev/issue/74609
 - change to the garbage collector that allows it detect some types of goroutine leaks (a.k.a. partial deadlocks in the academic community)

Accepted

- crypto/mldsa: new package go.dev/issue/77626
 - mldsa implements the post-quantum ML-DSA signature scheme specified in FIPS 204
- spec: type inferred composite literals go.dev/issue/12854
 - allows composite literals to omit the type when it can be inferred from the context
- cmd/go: change `go mod init` default go directive back to 1.N go.dev/issue/77653

.http



github.com/oktalz/dot-http

tool for executing `.http` files

This project includes two components:

- **CLI tool** (`dot-http`) — run `.http` files from the terminal
- **VSCode extension** — run `.http` files directly inside VS Code
 - see [dot-http VSCode extension](#)

Installation:

```
go install github.com/oktalz/dot-http@latest
```

Usage:

```
dot-http [flags] <file.http> [request-name]
```


Flag

Description

<code>-v</code>	Print version and exit
<code>-V</code>	Verbose: print request details before sending
<code>-H</code>	Print response headers only (no body)
<code>-B</code>	Print response body only (no headers)
<code>-o <file></code>	Save response body to a file
<code>--env <name></code>	Load <code><name>.env</code> instead of <code>.env</code>
<code>--timeout <dur></code>	Request timeout, e.g. <code>30s</code> , <code>1m</code> (default: none)
<code>--no-follow</code>	Do not follow redirects
<code>--session <file></code>	Load/save cookies from/to a JSON session file
<code>--junit <file></code>	Write JUnit XML test report to file

format - file structure

- Requests are separated by `###` .
- Each block can have headers

```
GET https://api.example.com/users
```

```
###
```

```
POST https://api.example.com/users  
Content-Type: application/json
```

```
{  
    "name": "John"  
}
```

format - variables

File variables — defined at the top of the `.http` file:

```
@baseUrl = https://api.example.com
```

```
###
```

```
GET {{baseUrl}}/users
```

Environment variables — from system environment and a `.env` file

```
API_URL=https://api.example.com
```

```
TOKEN="secret"
```

```
GET {{API_URL}}/users
```

```
Authorization: Bearer {{TOKEN}}
```



Multiple environments

Use `--env` to load a named environment file:

```
dot-http --env prod api.http      # loads prod.env
dot-http --env staging api.http    # loads staging.env
```

format - request chaining

Name a request with `# @name` and reference its response in requests using dot notation:

```
# @name = login
POST https://api.example.com/auth
Content-Type: application/json

{"username": "user", "password": "pass"}

###

# @requires = login
GET https://api.example.com/profile
Authorization: Bearer {{login.body.token}}
```

Available response fields:

- `{{name.body.field}}` — JSON response body (dot notation for nested fields)
- `{{name.headers.Content-Type}}` — response headers



format - parameterized dependencies

Pass arguments to dependencies to reuse the same request with different parameters:

```
# @name = getUser
GET https://api.example.com/users/{{id}}

###

# @requires = getUser(id=1)
# @requires = getUser(id=2)
GET https://api.example.com/compare
```



format - assertions & testing

Use `# @expect` to assert the response status code, and `# @assert` for header/body checks. Exit code is non-zero on failure — suitable for CI.

```
# @name = createUser
# @expect 201
# @assert body.name == John
# @assert header.Content-Type contains application/json
POST https://api.example.com/users
Content-Type: application/json

{"name": "John"}
```

Supported operators: `==`, `!=`, `contains`, `!contains`



format - JUnit XML reports

- Generate a JUnit-compatible XML report for CI/CD systems
 - GitHub Actions, Jenkins, etc.

```
dot-http --junit report.xml api.http
```


format - GraphQL

Set `Content-Type: application/graphql` and write the query as the body

- it will be automatically wrapped as `{"query": "..."}`

```
POST https://api.example.com/graphql
Content-Type: application/graphql
```

```
{
  users {
    id
    name
  }
}
```

format - cookies

Persist cookies between runs using a session file:

```
dot-http --session session.json api.http login  
dot-http --session session.json api.http profile
```

- Cookies from the `login` request will be saved to `session.json` and loaded for the `profile` request.



format - imports

Split requests across files using `@import` .

- Imported files make their named requests and variables available to the current file.

```
@import = auth.http
@import = helpers/users.http

###
# @requires = login
GET https://api.example.com/profile
Authorization: Bearer {{login.body.token}}
```

- Paths are relative to the importing file's directory
- Multiple `@import` directives are supported
- Imports are recursive (imported files can import other files)
- Cyclic imports are detected and safely skipped



OAuth2

Declare OAuth2 directives as per-request comments.

- The token is acquired automatically and injected as `Authorization: Bearer <token>`.
- Tokens are cached with expiry so re-runs don't re-authenticate.

Types:

- Client Credentials (machine-to-machine)
- Authorization Code + PKCE (browser flow)
- Device Code Flow

Client Credentials (machine-to-machine)

```
# @oauth2-grant = client_credentials
# @oauth2-token-url = https://auth.example.com/oauth/token
# @oauth2-client-id = {{CLIENT_ID}}
# @oauth2-client-secret = {{CLIENT_SECRET}}
# @oauth2-scope = read write
GET https://api.example.com/resource
```

Authorization Code + PKCE (browser flow)

- Spins up a local server, opens the browser, captures the code, exchanges it for a token.

```
# @oauth2-grant = authorization_code
# @oauth2-token-url = https://auth.example.com/oauth/token
# @oauth2-auth-url = https://auth.example.com/oauth/authorize
# @oauth2-client-id = {{CLIENT_ID}}
# @oauth2-scope = openid profile
# @oauth2-redirect-port = 9876
GET https://api.example.com/resource
```



Device Code Flow

- Prints the verification URL and user code, then polls until approved.

```
# @oauth2-grant = device_code
# @oauth2-token-url = https://auth.example.com/oauth/token
# @oauth2-device-url = https://auth.example.com/oauth/device/
# @oauth2-client-id = {{CLIENT_ID}}
# @oauth2-scope = read
GET https://api.example.com/resource
```

format - OAuth2

Token caching

Add `@oauth2-cache` at the file level to persist tokens across runs:

```
@oauth2-cache = .tokens.json
```



```
# Run a specific named request (and its dependencies)
```

```
dot-http api.http login
```

```
# Verbose: show request details before sending
```

```
dot-http -V api.http login
```

```
# Headers only
```

```
dot-http -H api.http login
```

```
# Body only (useful for piping to jq, etc.)
```

```
dot-http -B api.http login | jq .
```

```
# Save response body to file
```

```
dot-http -o response.json api.http
```

```
# Set a request timeout
```

```
dot-http --timeout 10s api.http slow_request
```

VSCode extension

```
test.http
1  @URL2 = https://jsonplaceholder.typicode.com
2  @createdAt = {{$datetime iso8601}}
3  @modifiedBy = {{$env USERNAME}}
4
5  ###
6  # @name = first_request
   Run Request
7  GET {{$URL}}/posts/1?username={{USERNAME}}
8
9  ###
10
11 # @name = second_request
12 # @requires = first_request
   Run Request
13 GET https://jsonplaceholder.typicode.com/posts/{{first_request.body.userId}}
14
15 ###
16
17 # @name = third_request
18 # @requires = second_request
```



VSCode extension

```
! route.yaml {} settings.json a.present test.http X ... response: http X
```

```
test.http
1 @URL = https://jsonplaceholder.typicode.com
2 @createdAt = {{$datetime iso8601}}
3 @modifiedBy = {{$env USERNAME}}
4
5 ###
6 # @name = first_request
7 Run Request
8 GET {{URL}}/posts/1?username={{USERNAME}}
9
10 ###
11 # @name = second_request
12 # @requires = first_request
13 Run Request
14 GET https://jsonplaceholder.typicode.com/posts/{{first_request.body.id}}
15
16 ###
17 # @name = third_request
18 # @requires = second_request
19
20 Run Request
21 POST https://jsonplaceholder.typicode.com/posts
22 Content-Type: application/json
23 {
24   "title": "foo",
25   "data": "{{second_request.body.title}}",
26   "body": "bar2",
27   "userId": 2
28 }
29
```

```
1 HTTP/1.1 200 OK
2 date: Tue, 28 Apr 2026 08:45:31 GMT
3 content-type: application/json; charset=utf-8
4 content-length: 292
5 connection: close
6 access-control-allow-credentials: true
7 cache-control: max-age=43200
8 etag: W/"124-yikDLzqO5gfBrJFrcdJ8Yq0LGnU"
9 expires: -1
10 nel: {"report_to": "heroku-nel", "response_headers": ["Via"], "max_age": 1209600}
11 pragma: no-cache
12 report-to: {"group": "heroku-nel", "endpoints": [{"url": "https://nel.heroku.com/reports"}]}
13 reporting-endpoints: heroku-nel="https://nel.heroku.com/reports"
14 server: cloudflare
15 vary: Origin, Accept-Encoding
16 via: 2.0 heroku-router
17 x-content-type-options: nosniff
18 x-powered-by: Express
19 x-ratelimit-limit: 1000
20 x-ratelimit-remaining: 977
21 x-ratelimit-reset: 1776759009
22 age: 1849
23 accept-ranges: bytes
24 cf-cache-status: HIT
25 server-timing: cfCacheStatus;desc="HIT", cfEdge;dur=17,cfOrigin;dur=0
26 cf-ray: 9f34ca8d8fa1ec28-ZAG
27 alt-svc: h3=":443"; ma=86400
28
29 {
30   "userId": 1,
31   "id": 1,
32   "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",
33   "body": "quia et suscipit suscipit reprehenderit ea mollitia tenetur similique id neque reiciendis totam recusandae sit aspernatur nemo. In harum veniam qui. vero voluptas accusantium aliquam accusantium perferendis. nam aut fuga expedit aut repellat."
34 }
```



VSCode extension

User Workspace Last synced: 0 secs ago

▼ Extensions

Gemini CLI

Gemini Code Assist

GHE.com & GitHub En...

Git

Git History

GitHub

GitHub Actions

GitHub Copilot Chat

> GitLens — Git superch...

Go

Go Template Support

Graphviz Configuration

Graphviz Interactive P...

Grunt

Gulp

HTML

Http Client

isort

Jake

> JavaScript and TypeSc...

JavaScript Debugger

JavaScript Test Runner

Extensions

Http Client

Follow Redirects

☒ Automatically follow HTTP redirects.

OAuth2 Cache File

File to cache OAuth2 tokens (relative to workspace root). Used when @oauth2-cache is not set in the .http file.

`.tokens.json`

Response View Mode

Where to show the response: in a new tab, reuse the same tab, or in the output channel.

`reuseTab`

Timeout

Request timeout, e.g. '30s', '1m', '500ms'. Empty means no timeout.

